

# AI and ML for Architecture (M25RD204)

Dr. Nimrita Koul

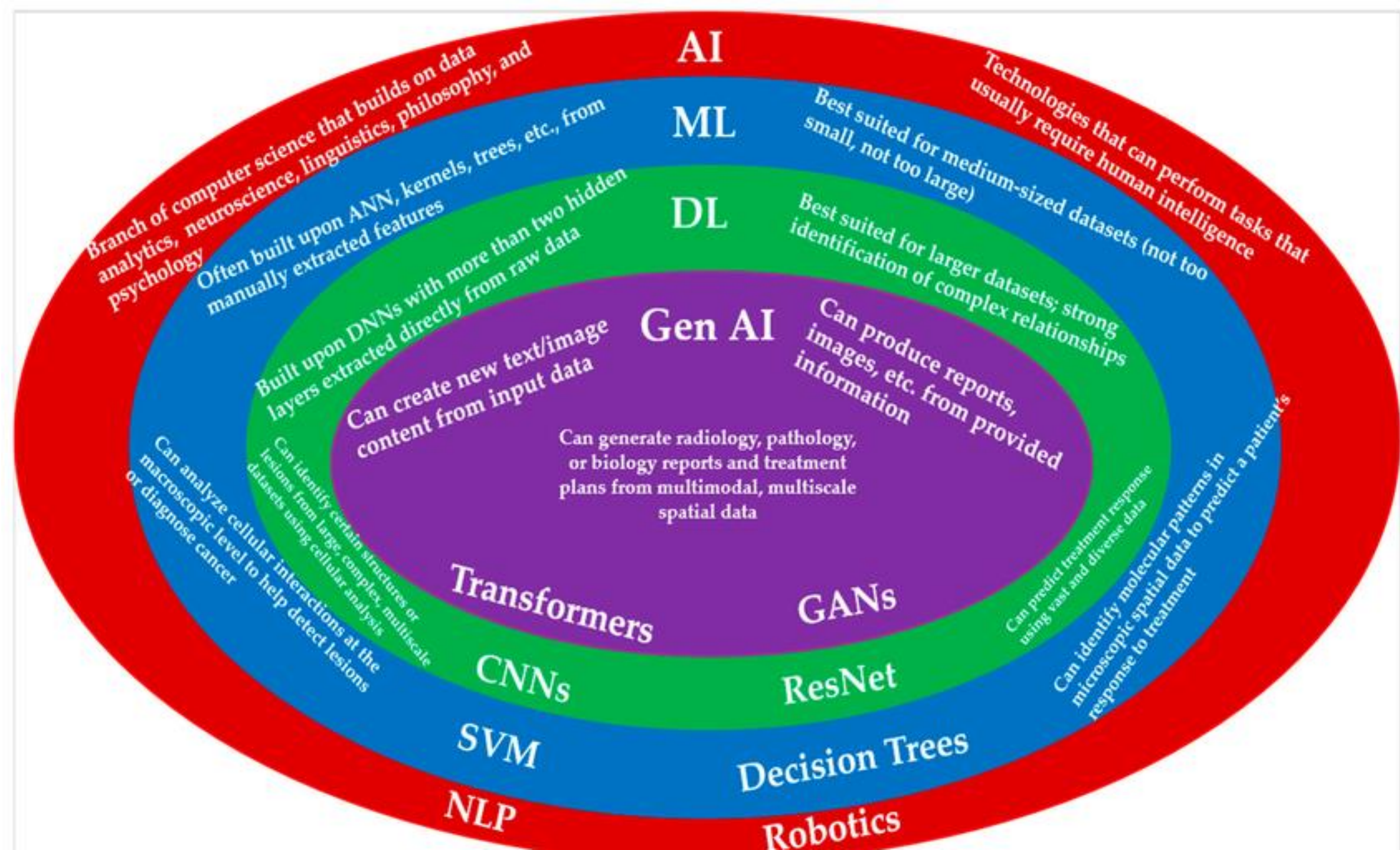
Associate Professor, School of CSE, REVA University

# Course Contents & Schedule

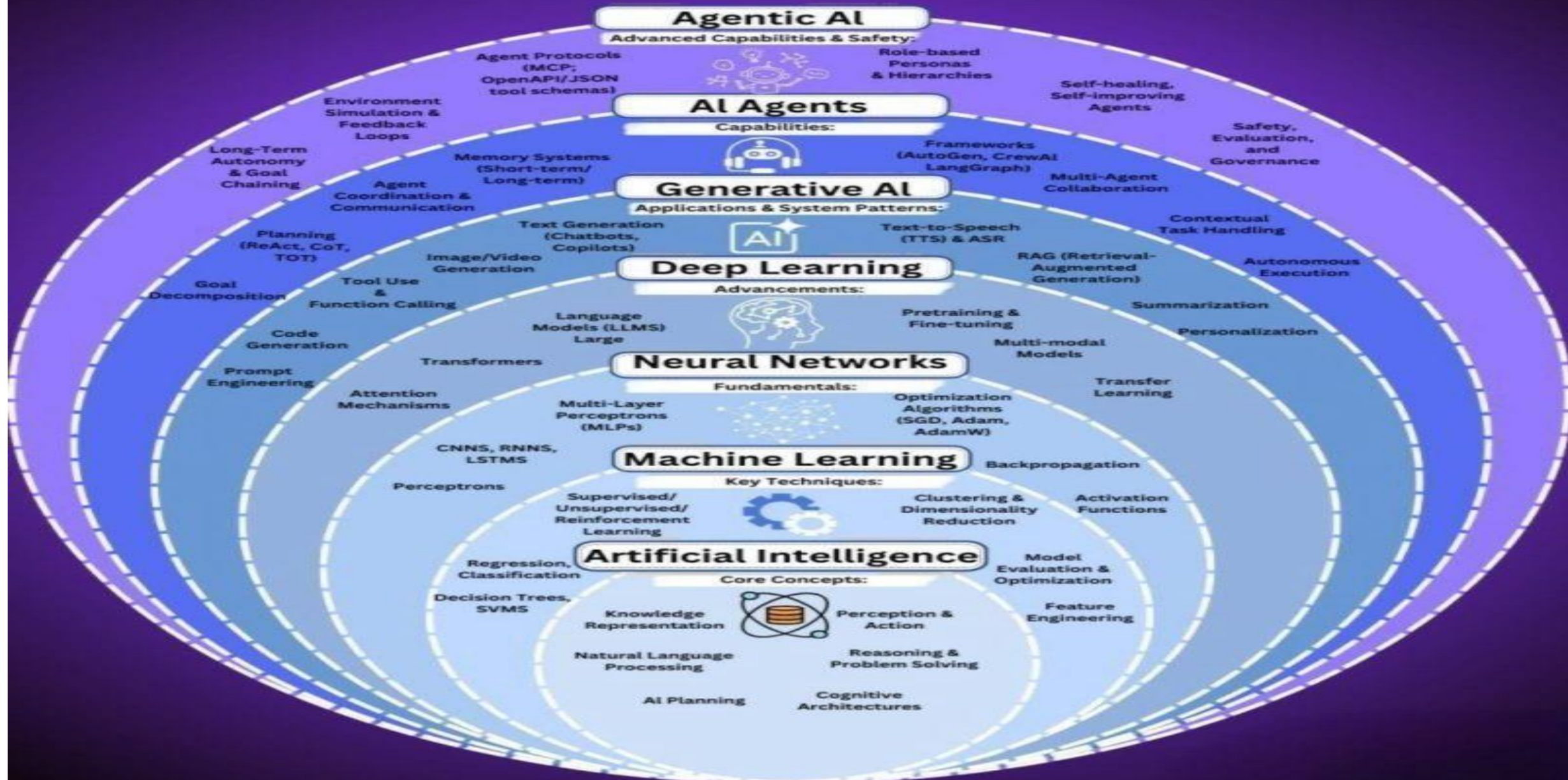
1. Python Programming + Relevant Libraries : 1.7.2026
2. Introduction to AI, Data Science and ML – Data Collection and Preprocessing : 2.7.2026
3. Supervised Learning Algorithms – Linear Regression for Space Planning : 2.7.2026
4. Unsupervised Learning – K - Means for Site Analysis : 2.7.2026
5. Neural Networks, Deep Learning : 6.7.2026
6. Computer Vision – CNNs, Image Classification, Stable Diffusion, Mid Journey : 6.7.2026
7. Generative Design with GANs : 7.7.2026
8. Generative AI – LLMs, VLMs, Multimodal AI Models : 7.7.2026
9. Generative AI Tools for Architecture : 7.7.2026

# Course Contents

- 10. Natural Language Processing for Design Briefs – Text Analysis and Brief Processing : 8.7.2026
- 11. Reinforcement Learning for Space Utilization : 8.7.2026
- 12. Optimization Algorithms – Genetic Algorithm for Layout Planning : 9.7.2026
- 13. Building Performance Prediction – Energy Consumption ML Models : 13.7.2026
- 14. Smart Building Systems – IoT Integration with ML Planning : 13.7.2026
- 15. Advanced Applications - Custom AI Tool Development : 14.7.2026
- 16. AI Ethics in Architecture – Bias Detection and Mitigation : 14.7.2026
- 17. Industry Case Studies – Portfolio Compilation: 14.7.2026
- 18. Project Presentations – 15.7.2026



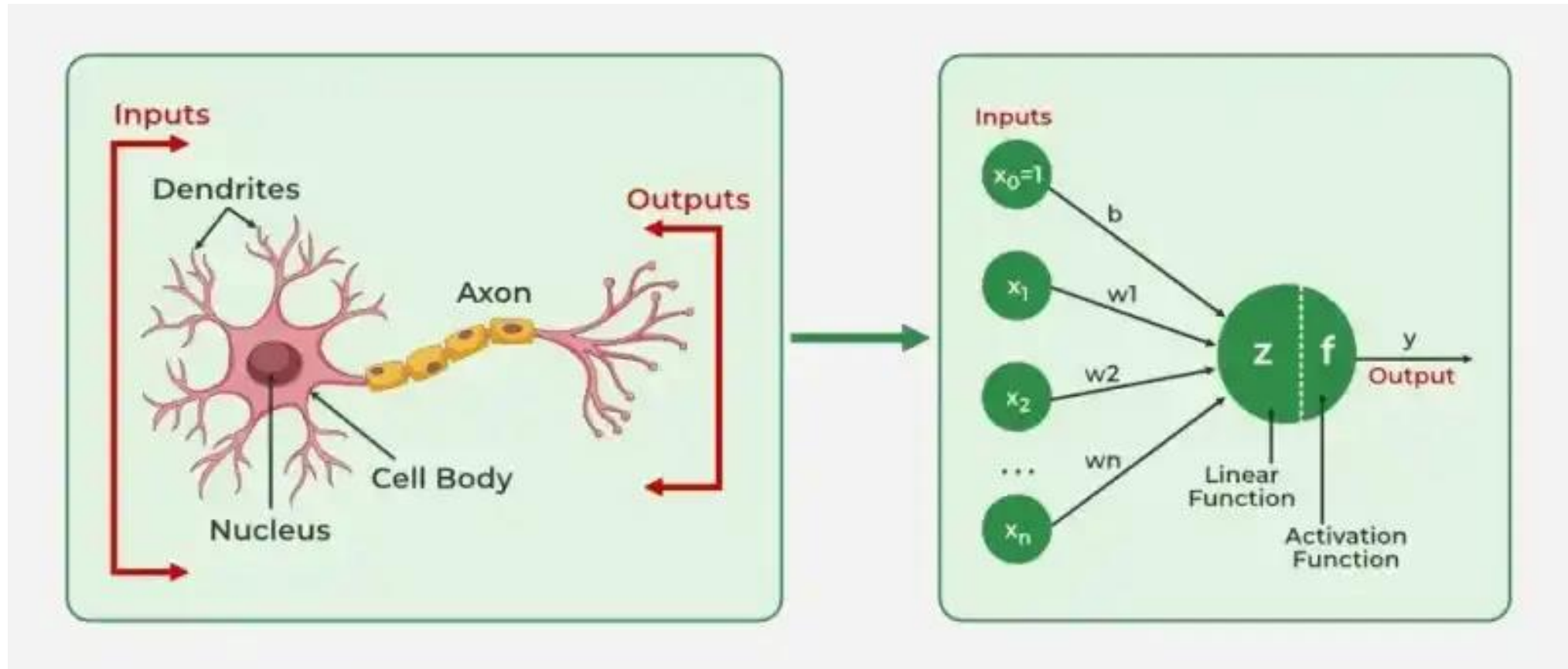
# The Agentic AI Universe



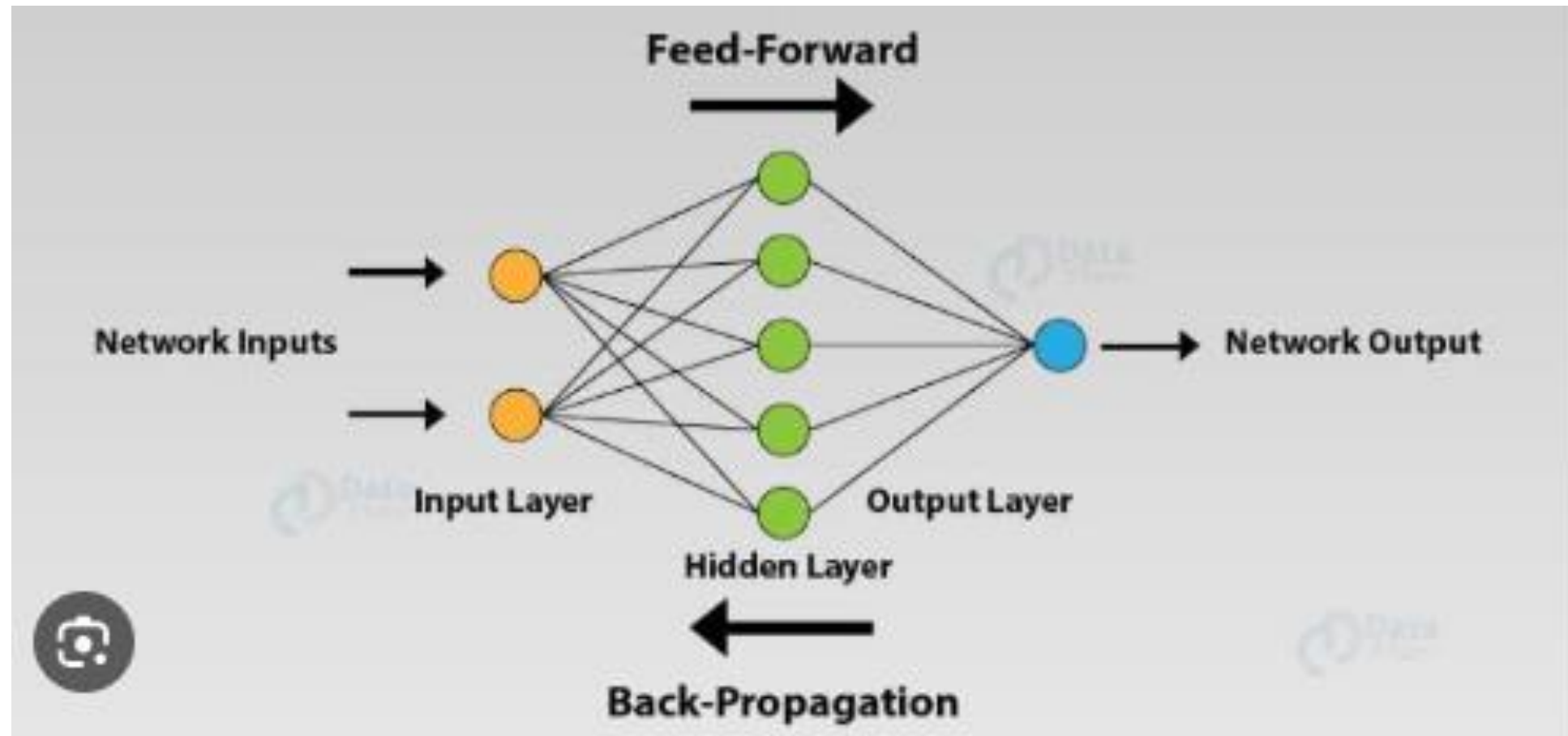
# Deep Learning

- Deep Learning (DL) is a subset of Machine Learning (ML) that uses multi-layered artificial neural networks to learn representations of data with multiple levels of abstraction.
- Unlike traditional ML, which requires human experts to manually engineer and select features (e.g., explicitly defining how to measure a crack in concrete), Deep Learning algorithms perform **automatic feature extraction**. They learn to identify relevant patterns directly from raw data (such as images, point clouds, or sensor logs).

# Biological Neuron and Artificial Neuron

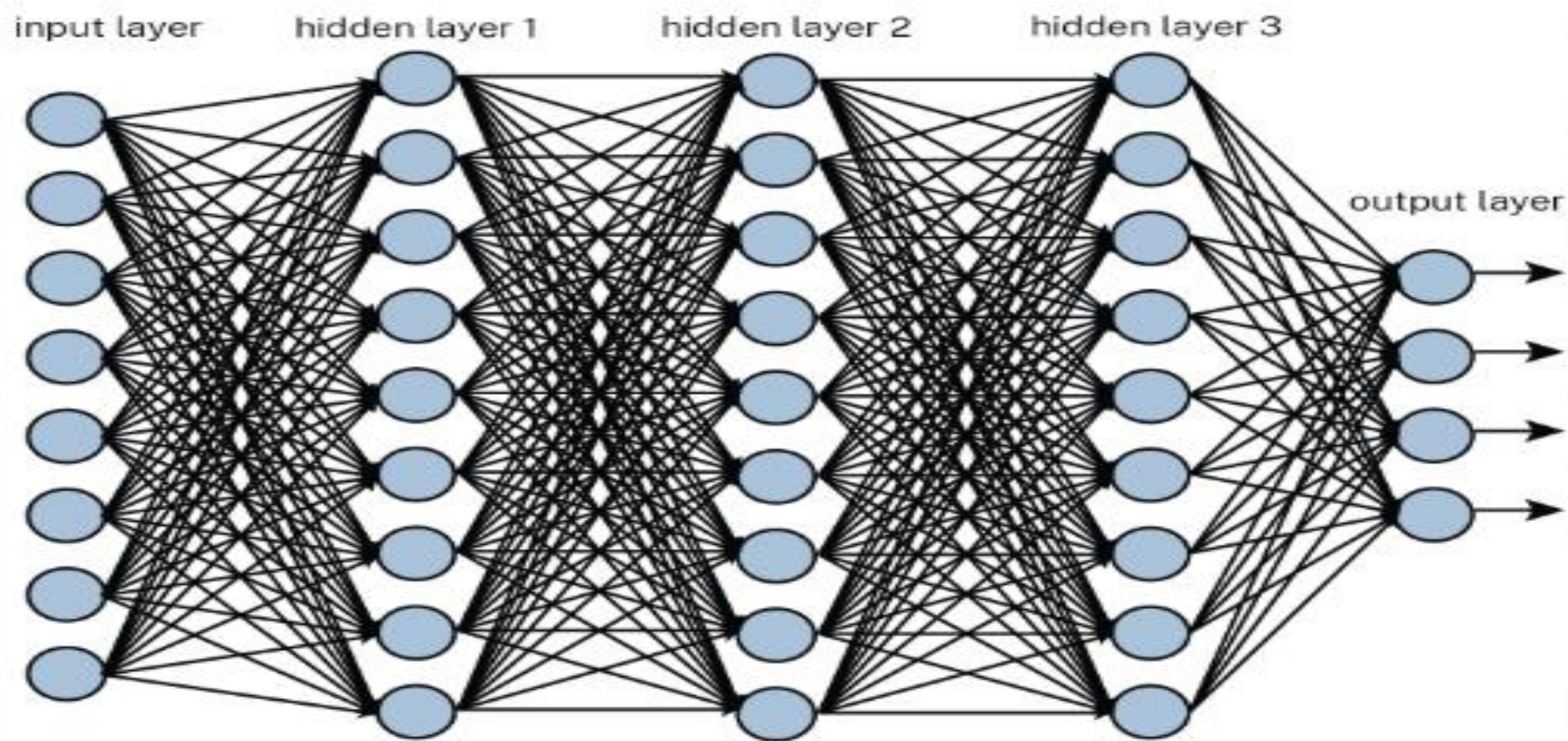


# Artificial Neural Network (ANN)



# Artificial Neural Network (ANN)

- An Artificial Neural Network is a computational model inspired by the structure and function of biological neural networks in the brain.
- It consists of interconnected nodes called **neurons** organized into three distinct layer types:
- **Input Layer:** Receives the raw data (e.g., pixel values of an architectural drawing or drone image).
- **Hidden Layer(s):** Performs nonlinear transformations on the data. A network is considered "deep" if it contains multiple hidden layers.
- **Output Layer:** Produces the final prediction (e.g., "Structural Crack Detected" or "No Damage").



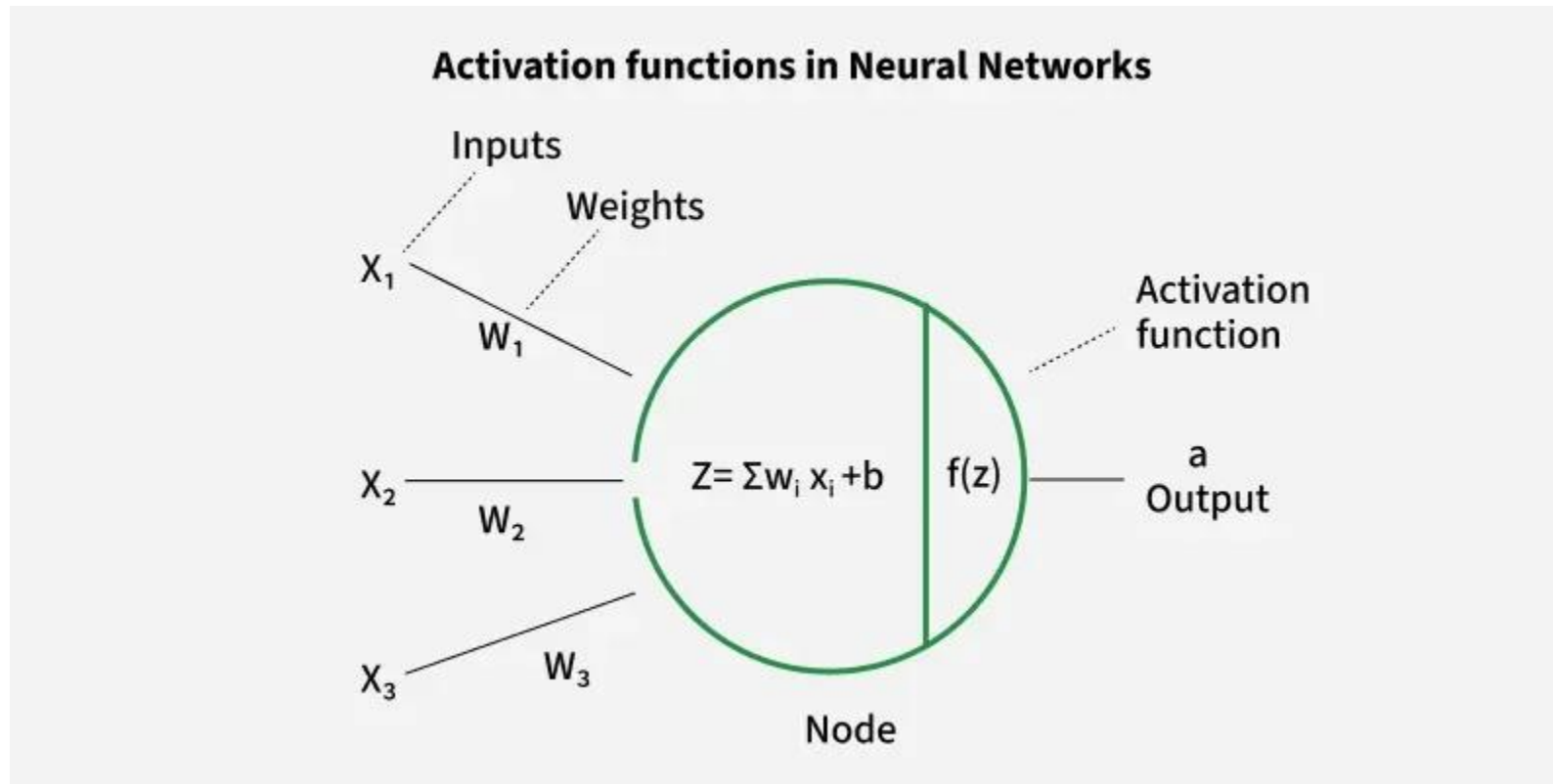
SOURCE: National Energy Research Scientific Computing Center document Peter Harrington and Steven Farrell, "Scientific Deep Learning on Perlmutter," National Energy Research Scientific Computing Center, January 7, 2022, p. 3, <https://www.nersc.gov/assets/Uploads/Deep-Learning-on-Perlmutter.pdf>.

NOTE: Adapted from the original by National Energy Research Scientific Computing Center.

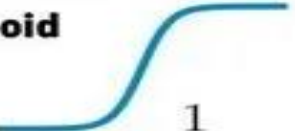
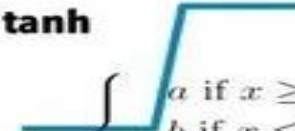
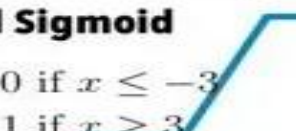
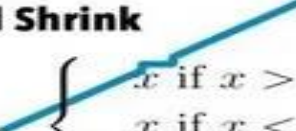
# Model Parameters

- Each connection between neurons has an associated **weight** (strength of the connection) and a **bias** (offset). During training, the network adjusts these weights and biases using backpropagation to minimize the error between its predictions and the ground truth.

# Activation Functions



## Neural Network Activation Functions: a small subset!

|                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ReLU</b><br><br>$\max(0, x)$                                                                                                        | <b>GELU</b><br><br>$\frac{x}{2} \left( 1 + \tanh \left( \sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$                                                      | <b>PReLU</b><br><br>$\max(0, x)$                                                                                                          |
| <b>ELU</b><br><br>$\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$                          | <b>Swish</b><br><br>$\frac{x}{1 + \exp -x}$                                                                                                                   | <b>SELU</b><br><br>$\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$                                                                      |
| <b>SoftPlus</b><br><br>$\frac{1}{\beta} \log(1 + \exp(\beta x))$                                                                       | <b>Mish</b><br><br>$x \tanh \left( \frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$                                                                           | <b>RReLU</b><br><br>$\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$   |
| <b>HardSwish</b><br><br>$\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$ | <b>Sigmoid</b><br><br>$\frac{1}{1 + \exp(-x)}$                                                                                                                | <b>SoftSign</b><br><br>$\frac{x}{1 +  x }$                                                                                                |
| <b>Tanh</b><br><br>$\tanh(x)$                                                                                                         | <b>Hard tanh</b><br><br>$\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$                               | <b>Hard Sigmoid</b><br><br>$\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x > 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$  |
| <b>Tanh Shrink</b><br><br>$x - \tanh(x)$                                                                                             | <b>Soft Shrink</b><br><br>$\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$ | <b>Hard Shrink</b><br><br>$\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$ |

# Neural Network Playground

- <https://playground.tensorflow.org/>
- <https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=circle&regDataset=reg-plane&learningRate=0.03&regularizationRate=0&noise=0&networkShape=4,2&seed=0.72316&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false>
- <https://www.ccom.ucsd.edu/~cdeotte/programs/neuralnetwork.html>
- Simple Classifiers:
- <https://www.ccom.ucsd.edu/~cdeotte/programs/classify.html>

# Core Types of Neural Networks

Different network architectures are optimized for different types of spatial and architectural data:

| Network Type                                  | Best Suited For                        | Architectural/Spatial Application                                                              |
|-----------------------------------------------|----------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Artificial Neural Networks (ANN / MLP)</b> | Tabular, structured data               | Predicting building energy consumption, real estate valuation, project cost estimation.        |
| <b>Convolutional Neural Networks (CNN)</b>    | Grid-like topology (Images, 2D planes) | Structural damage inspection, automated floor plan classification, satellite site analysis.    |
| <b>Recurrent Neural Networks (RNN / LSTM)</b> | Sequential or time-series data         | Sensor data logs over time, HVAC optimization schedules, pedestrian movement patterns.         |
| <b>Graph Neural Networks (GNN)</b>            | Non-Euclidean data (Graphs, networks)  | Spatial topological layouts (rooms as nodes, corridors as edges) for automated space planning. |

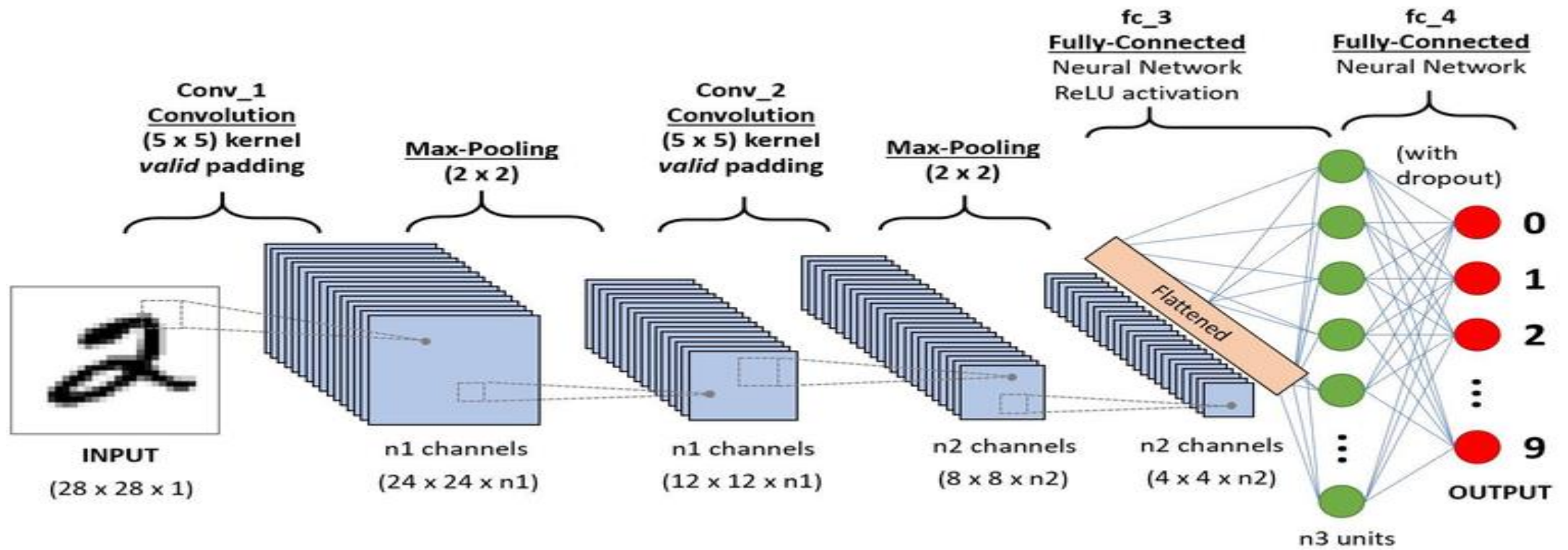
# Convolutional Neural Networks (CNNs)

- A Convolutional Neural Network (CNN) is a class of deep neural networks most commonly applied to analyzing visual imagery.
- Standard ANNs fail on images because treating an image as a flat vector of pixels destroys spatial hierarchy and results in a massive explosion of trainable parameters. CNNs solve this by enforcing a local connectivity pattern. They preserve the spatial relationship between pixels, making them ideal for spatial design, mapping, and site inspection.

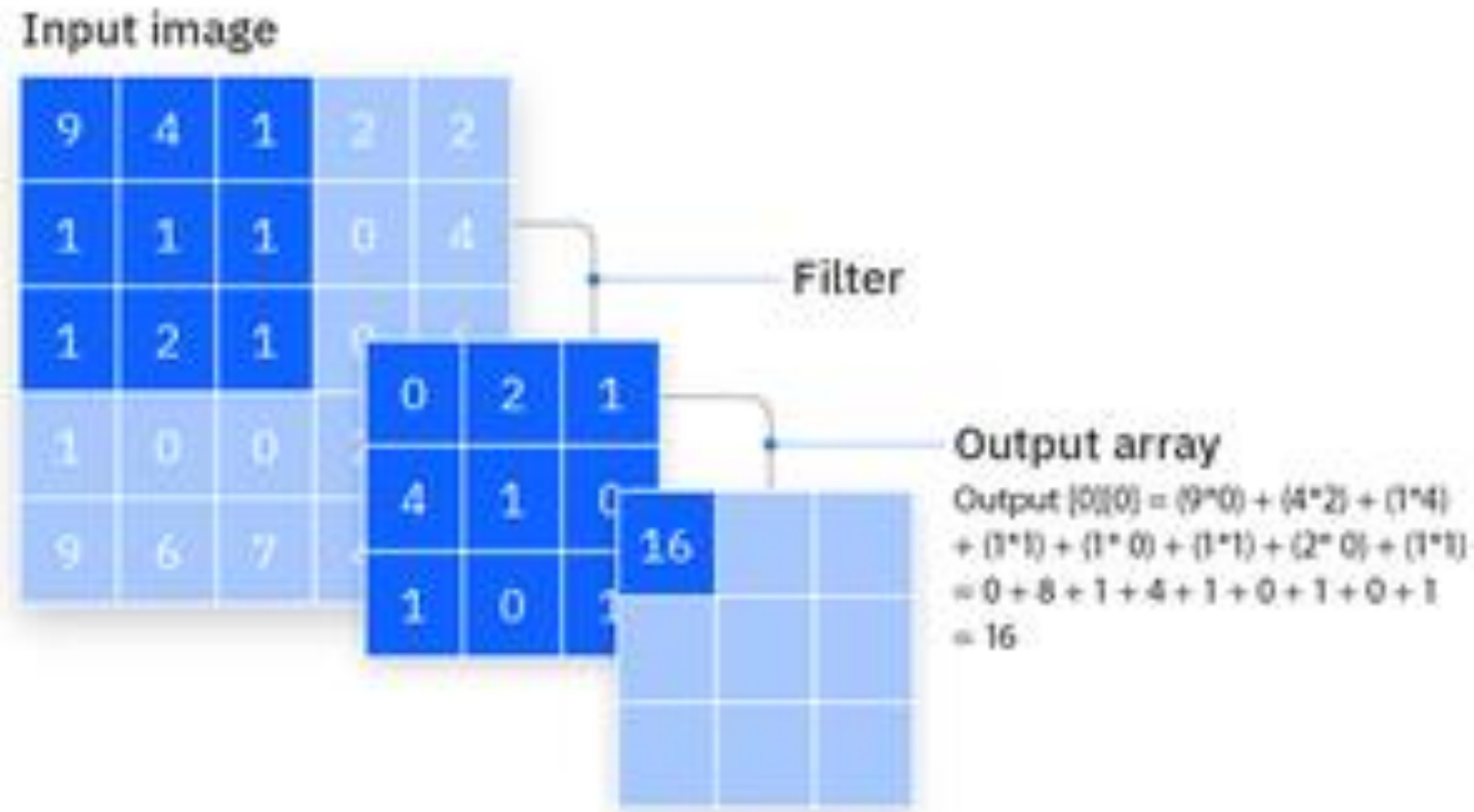
# Key Layers in a CNN

- A CNN processes an image through a pipeline of specialized layers:
- **Convolutional Layer:** This is the core building block. It applies a series of small, learnable filters (kernels) that slide (convolve) across the input image. These filters detect local features such as edges, textures, and gradients.
- **Activation Function (ReLU):** Introduces non-linearity to the network, allowing it to learn complex, non-linear boundaries in the visual data.
- **Pooling Layer (Max Pooling):** Reduces the spatial dimensions (width and height) of the data. This downsampling retains the most critical feature information while reducing computational load and making the network invariant to small translations/shifts in the image.
- **Fully Connected Layer:** Sits at the end of the network. It takes the high-level features extracted by the convolutional layers and flattens them to perform the final classification task.

# CNN



# Convolution Operation



# Case Study: Structural Damage Detection via Drone Imagery

- [Drone Capture] → [Image Preprocessing] → [CNN Feature Extraction] → [Localization & Classification]

# Steps

- **Data Acquisition (Drone Capture):** Unmanned Aerial Vehicles (UAVs) fly pre-programmed grids around a building envelope or structure, capturing high-resolution imagery from angles difficult or dangerous for human inspectors to reach.
- **Preprocessing:** Images are tiled, normalized, and augmented (rotated, scaled) to prepare them for the neural network.
- **CNN Feature Extraction:** \* The **early layers** of the CNN detect basic structural elements: straight lines, changes in contrast, and material edges.
- The **deeper layers** combine these edges to identify complex pathologies: spalling (flaking concrete), moisture entry, or structural cracks.
- **Classification & Localization:** The network outputs both a classification (e.g., "Crack Detected") and a bounding box or pixel mask (Semantic Segmentation) showing exactly where the damage exists on the building facade.